

5.4 Spring Security Web Config - SecurityFilterChain Configuration

• What Is SecurityFilterChain ?

- **SecurityFilterChain** is Spring Security's mechanism for defining security rules for incoming HTTP requests. Think of it as the **security policy** of web applications.
- It answers questions such as:
 - Which URLs are **public**?
 - Which URLs **require authentication**?
 - Which URLs **require ADMIN**?
 - How should users **log in**?
 - Should the application use **sessions**?
 - Should **CSRF protection** be enabled?
 - What should happen when **authentication fails**?
 - What should happen when **authorization fails**?

• Why Is It Called a “Filter Chain” ? :

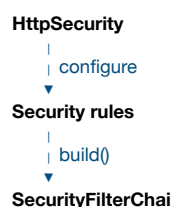
- Spring Security is heavily based on **Servlet filters**.
- Instead of: **Request** → **Controller** the request goes through **security filters** first:
 - **Request** → **Filter-1** → **Filter-2** → **Filter-3** → **Filter-4** → ... → **Controller**
- **Spring Security** combines these **filters** into a **SecurityFilterChain** and each filter has a particular **responsibility**.
- For example, depending on the configuration, filters may deal with:
 - Authentication
 - Authorization
 - CSRF
 - Session management
 - Security context
 - Bearer tokens
 - Exception handling
 - Login/logout
- **Spring Security** doesn't secure the application with one giant class. It uses a **chain** of **filters**, each **responsible** for a particular security concern.

• Why Do We Configure SecurityFilterChain ? :

- Modern Spring Security applications commonly define a bean like:

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) {
    ...
}
```

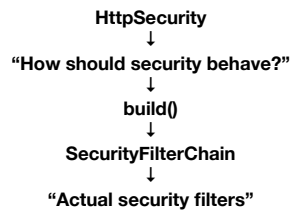
- The important object here is **HttpSecurity**, It is a **builder/configuration** API using this we can tell Spring: **“For HTTP requests, I want these security rules”**. And eventually: **return http.build()**; creates the actual **SecurityFilterChain**.
- So conceptually:



5.4 Spring Security Web Config - SecurityFilterChain Configuration

• HttpSecurity vs SecurityFilterChain :

- **HttpSecurity:** It is used to configure web security.
- **SecurityFilterChain:** The resulting filter chain used by Spring Security.
- It is like this:



• .authorizeHttpRequests() :

- It is one of the most important method in HttpSecurity, By using this we can set Which requests are allowed to which users.
- Example:

```
httpSecurity
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/posts/**").hasRole("ADMIN") // only ADMIN can access — /post, /post/x, /post/y, ...
    .requestMatchers("/profile").authenticated() // user need to be logged-in before accessing /profile
    .requestMatchers("/public/**").permitAll()); // anyone can access — /public, /public/x, /public/y, ...
```

• .requestMatchers() :

- **requestMatchers()** determines which requests a rule applies to.
- You can match based on:
 - URL patterns
 - HTTP methods
 - Other request characteristics depending on the matcher
- Example:

```
.requestMatchers(HttpMethod.GET, "/posts/**")
.hasAuthority("POST_READ")

.requestMatchers(HttpMethod.DELETE, "/posts/**")
.hasAuthority("POST_DELETE")
```

• '*' vs '**'

- **'*'** Generally matches within a **single path segment** like **'/posts/*'** matches with: **/post/1** or **/posts/2** or **/posts/3** but not necessarily with **/posts/1/comments** .
- **'**'** Can match multiple path segments like **'/posts/**'** matches with:
 - **/posts**
 - **/posts/1**
 - **/posts/comments**
 - **/posts/comments/1**
 - **/posts/comments/1/replies**

5.4 Spring Security Web Config - SecurityFilterChain Configuration

• authenticated() :

- It means the request is allowed only if the user has been **successfully authenticated**. It doesn't care whether the user is **ADMIN** or **USER**.
- For `.requestMatchers("/profile").authenticated()`; Both USER and ADMIN can access **/profile**, assuming both are authenticated.

• permitAll() :

- `.permitAll()` means anyone can access this endpoint **without authentication**. Like:
 - /login
 - /register
 - /public/**
 - /error
- Example:

```
httpSecurity
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/register", "/login").permitAll());
```

• hasRole() :

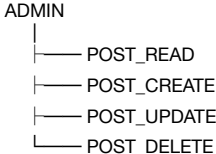
- `.hasRole("ADMIN")` means the authenticated user must have the **ADMIN** role.
- `.hasRole("ADMIN")` normally corresponds to: **ROLE_ADMIN**, Spring Security's role convention adds the **ROLE_** prefix.
- Then Why do we write ADMIN instead of ROLE_ADMIN? Because `hasRole("ADMIN")` is effectively checking for the authority: **ROLE_ADMIN**

• hasAuthority() :

- `hasAuthority()` is used to **authorize** access based on a specific **GrantedAuthority**. It checks whether the currently authenticated user has the exact authority specified.
- For example, `.requestMatchers("/posts/**").hasAuthority("POST_READ")`, this means only a user who has the exact **POST_READ** authority can access this endpoint.
- Unlike `hasRole()`, `hasAuthority()` does not add anything like **ROLE_** prefix. It checks the authority name exactly as specified.

• Roles vs Authorities :

- Role Represents a broad category of user. Like: **ROLE_USER**, **ROLE_ADMIN**, **ROLE_MANAGER**.
- Authority Represents a specific permission. **POST_READ**, **POST_CREATE**, **POST_UPDATE**, **POST_DELETE**.
- Example:



- **Roles** are convenient for **coarse-grained** authorization where **Authorities** are useful for **fine-grained** authorization.

5.4 Spring Security Web Config - SecurityFilterChain Configuration

• hasAnyRole() :

- Sometimes multiple roles should be allowed. `requestMatchers().hasAnyRole("ADMIN", "MANAGER")` Means both ADMIN and MANAGER are allowed but USER is not.
- Similarly `requestMatchers().hasAnyAuthority("POST_READ", "POST_ADMIN")` can be used for authorities.

• anyRequest() :

- It matches all requests which weren't already matched by a previous rule. For example:

```
httpSecurity
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/public/**").permitAll()
    .requestMatchers("/posts/**").hasRole("ADMIN")
    .anyRequest.authenticated() ; // Think of it as the fallback rule.
```

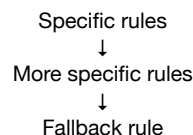
• Rule Ordering Matters :

```
httpSecurity
.authorizeHttpRequests(auth -> auth
    .anyRequest.authenticated()
    .requestMatchers("/admin/**").hasRole("ADMIN");
```

- Conceptually this is **wrong** because the broad rule comes first.
- Correct approach is :

```
httpSecurity
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .anyRequest.authenticated());
```

- Think it like as :



- Put specific authorization rules **before** broad **catch-all** rules.

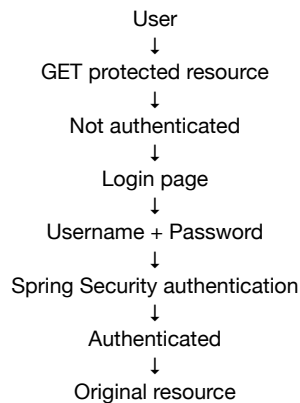
• Authentication and Authorization are Different :

- Suppose admin logs in successfully. Authentication: "Is this really admin?" —> YES.
- Then a request arrives `/posts/1` and Authorization asks: "Does admin have permission to access this?" —> `ROLE_ADMIN`? —> YES.
- So,
 - Authentication = Identity
 - Authorization = Permission

5.4 Spring Security Web Config - SecurityFilterChain Configuration

• formLogin() :

- formLogin() is an authentication mechanism provided by Spring Security. It enables form-based authentication. If we don't create any login page, it provides a default one.
- Flow:



• formLogin() in REST APIs :

- A browser-oriented application might use the formLogin() method, but modern REST APIs commonly use JWT or Bearer tokens.

• httpBasic() :

- Another common mechanism is .httpBasic() which enables **HTTP Basic** authentication.
- The client sends credentials using the HTTP Authorization header.
- Conceptually: Authorization: Basic <encoded-credentials>
- It's useful for:
 - Simple internal APIs
 - Development/testing
 - Certain service-to-service scenarios
- But credentials must be protected with HTTPS.

• csrf() :

- CSRF stands for: **Cross-Site Request Forgery**. CSRF protection is another major part of web security configuration.
- We can disable CSRF by using .csrf(csrfConfig -> csrfConfig.disable()).
- Disabling CSRF is not the solution, For traditional browser applications using cookies/sessions, CSRF protection is important. For stateless APIs using bearer tokens in the Authorization header, the threat model is different.

• Session Management :

- Spring Security can manage authentication using HTTP sessions. A common configuration is:

```
httpSecurity
    .sessionManagement(sessionConfig -> sessionConfig.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
```

- One particularly important policy is: STATELESS

5.4 Spring Security Web Config - SecurityFilterChain Configuration

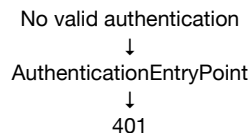
• Stateful vs Stateless :

	Stateful	Stateless
1	Server maintains the user's authentication/session state.	Server does not maintain a login session for each user.
2	A session is created after successful login.	No server-side session is required.
3	A unique Session ID is generated and stored, usually in a browser cookie.	A token (e.g., JWT/Bearer token) is sent with each request.
4	Subsequent requests use the Session ID to identify the authenticated user.	Each request contains the information required for authentication.
5	Server must store and manage session information.	Server validates the token for every request.
6	Session data is commonly stored in server memory or a session database/cache.	Authentication state is generally carried by the client in the token.
7	Can become difficult to scale across multiple servers without shared session storage or session affinity.	Easier to scale because servers do not need to share session state.
8	Logout can be handled by destroying or invalidating the server-side session.	Logout generally requires token expiry, revocation, or a token-management mechanism.
9	Common in traditional web applications using server-side sessions and cookies .	Common in REST APIs and JWT-based authentication .

• Exception thrown by spring security :

• **Authentication Failure — 401:**

- Suppose a user isn't authenticated and requesting for endpoint '*GET/posts/1*' which requires authentication.
- Then the appropriate HTTP response is typically **401 Unauthorized**.
- Conceptually:



• **Authorization Failure — 403:**

- Suppose the user is authenticated USER but requesting '*DELETE/posts/1*' but deletion requires *ROLE_ADMIN*.
- **Authentication succeeded** but **authorization failed**. Result: **403 Forbidden**.

• Meaning:

- 401 → "I don't know who you are."
- 403 → "I know who you are, but you're not allowed."

- These exceptions occur before the request reaches the controller (dispatcher servlet).

• exceptionHandling() :

- Spring Security provides `.exceptionHandling(...)` to customize **security exception** behavior.